PROJEKAT

Funkcionalna verifikacija hardvera

TEMA PROJEKTA:

Prepoznavanje objekata na slici

MENTOR PROJEKTA:

Nebojša Pilipović

PROJEKAT IZARDIO:

Nikola Kovačić EE 77/2020
Ognjen Višnjić EE 217/2020

U Novom Sadu, 10.9.2025.

Sadržaj

1. Uvod 3
2. Kreiranje verifikacionog plana 4
2.1 Opis rada dizajna 4
2.2 Opis nivoa verifikacije 5
3. Razvoj verifikacionog okruženja 5
3.1 Projektovanje sekvenci, paketa i sekvencera 5
3.2 Projektovanje driver-a 6
3.4 Projektovanje scoreboard-a 7
3.5 Projektovanje agenta i konfiguracije 8
3.6 Projektovanje interfejsa 8
3.7 Projektovanje test scenarija 8
3.8 Projektovanje okruženja 9
3.9 Projektovanje top modula 9
4. Prikupljanje pokrivenosti sistema 10
5. Zaključak 13

1. Uvod

Savremena IP jezgra predstavljaju veoma složene uređaje, i sastoje se iz ogromnog broja komponenti, u koje se ubrajaju registri, memorijski blokovi, i kombinaciona logika. Da bi se u potpunosti utvrdio ispravan rad takvih IP jezgara, neophodno je proveriti da li će sva moguća unutrašnja stanja i ulazni podaci uzrokovati generisanje ispravnog rešenja. Usled ograničene moći računara, takve provere su najčešće neisplative. Neophodno je uskladiti proces verifikacije sa finansijskim i vremenskim zahtevima, da bi se dobio proizvod traženog kvaliteta. Kao posledica, većina savremenih digitalnih uređaja poseduje greške koje nije lako pronaći.

Funkcionalna verifikacija je proces koji ima za cilj utvrđivanje da li će dizajn raditi u skladu sa zadatim funkcionalnim specifikacijama, pre nego što zamišljeni uređaj bude fabrikovan. Cena otklanjanja greške je višestruko manja ukoliko je greška otkrivena tokom verifikacije, nego prilikom testiranja proizvedenih uređaja. Proces funkcionalne verifikacije počinje od napisane funkcionalne specifikacije dizajna, i sastoji se iz šest koraka koji se zajedno zovu verifikacioni ciklus:

1. Kreiranje verifikacionog plana – on opisuje šta i na koji način treba da bude verifikovano. Verifikacioni plan se sastoji iz više poglavlja koja detaljno odgovaraju na pitanja kao što su: kakvi će se testovi i metode verifikacije koristiti, koji su softverski i hardverski resursi neophodni, koliko je verifikacionih inžinjera potrebno, kako raspodeliti njihov posao tokom vremena, koje funkcionalnosti treba da budu verifikovane, koji su uslovi završetka verifikacije.
2. Razvoj verifikacionog okruženja - predstavlja softverski kod koji emulira neposredno okruženje dizajna, i koje ima mogućnost provere ispravnosti rezultata. Razni alati olakšavaju proveru ispravnosti rada dizajna, sa već implementiranim funkcionalnostima kao što je analiza strukturne pokrivenosti. Verifikaciono okruženje se može razviti tako da omogućava različite metode verifikacije hardvera, u koje se ubrajaju okruženja zasnovana na zlatnim vektorima, referentnim modelima, deterministički, formalni, i drugi.
3. Debagovanje dizajna i verifikacionog okruženja – u toku izvršavanja testova se pronalaze razlike između očekivanih rezultata verifikacionog okruženja i stvarnih rezultata dizajna, sa ciljem da se utvrde i uklone uzroci tih razlika.
4. Izvršavanje regresionih testova - predstavlja ponovno izvršavanje testova nakon ispravljanja otkrivenih grešaka, zbog mogućeg nastanka novih problema.
5. Debagovanje proizvedenog hardvera - nakon fabrikacije hardvera, proces debagovanja je mnogo učinkovitiji zbog razlike u brzini između pravog hardvera i njegovog simulatora. Moguće je pronaći greške u ovom koraku, međutim primarni cilj je otkrivanje greški pre fabrikacije s obzirom na veliku cenu otklanjanja takvih greški.
6. Izvršavanje eskejp analize (eng. escape analysis) - ukoliko su uočene greške nakon fabrikacije, neophodno je utvrditi zbog čega nisu detektovane tokom simulacije, sa ciljem da bi se takvi propusti otklonili u budućim projektima.

Slika 1, Verifikacioni ciklus

U okviru ovog projekta vrši se verifikacija nad VHDL modulom *control_path*, koji implementira obradu slike na osnovu boje objekata. Verifikacija je rađena UVM metodologijom, gde se proverava ispravna vrednost objects_count_out i korektnu BRAM komunikaciju između DUT-a i testbenča.

Okruženje se sastoji od standardnih UVM komponenti: sequencer, driver, monitor i scoreboard. Testovi se izvršavaju u Vivado/XSim simulatoru, a rezultati pokrivenosti se izveštavaju kroz HTML izveštaj.

2. Kreiranje verifikacionog plana

2.1 Opis rada dizajna

Verifikovani modul *control_path* je kontrolna FSM koja upravlja obradom 24-bitne RGB slike radi brojanja objekata definisanih opsegom boje. Interfejs koristi signale za pokretanje i završetak obrade, dimenzije slike *rows_in/cols*_in 61x61, pragove *lower_in/upper_in*, dva memorijska porta BRAM1 i BRAM2 sa linearnim adresiranjem $addr = y*cols + x$ i izlaz objects_count_out.

Obrada se izvodi fazno. Najpre se slika učitava u BRAM1 i kreće prvi korak obrade slike gde se u BRAM2 formira osnovna binarna maska, piksel se postavlja kao beo ako su sve RGB komponente unutar zadatog opsega, a crn ako ne pripada tom opsegu. Zatim se vrši dilatacija maske kako bi se popunile uske praznine i spojili delovi istog objekta. Nakon toga sledi provera homogenosti, piksel ostaje beo samo ako je njegova 11×11 okolina potpuno bela, čime se uklanjaju izolovani pikseli. Između faza maska se prekopira iz BRAM2 u BRAM1 radi konzistentnog izvora podataka i jasnog toka.

U završnom prolazu maska se skenira radi računanja površine i objekat se priznaje ako je površina i širina između zadatih granica i tada se *objects_count_out* uvećava. FSM računa kašnjenja pri čitanju/pisanju BRAM-a i proverava da su adrese u granicama slike, zato je redosled koraka strogo bitan i lako se može proveriti u verifikaciji.

2.2 Opis nivoa verifikacije

U ovom projektu proverava se samo najviši modul *control_path*. Ceo tok obrade slike je u tom modulu, pravljenje maske na osnovu opsega boje , dilatacija, erozija i brojanje validnih objekata, pa dodatna verifikacija nižih delova nije potrebna.

Cilj verifikacije je da se potvrdi tačnost izlaza *objects_count_out*, ispravno ponašanje *start/ready* signala, kao i pravilno čitanje/pisanje i adresiranje memorije po šemi *y*cols + x*. Test se pokreće na realnoj slici 61×61 za već definisan opseg boja, uključujući granične slučajeve (potpuno crna/bela maska).

3. Razvoj verifikacionog okruženja

Okruženje je urađeno UVM metodologijom i prati uobičajenu podelu uloga. Test postavlja očekivanja (npr. koliko objekata treba dobiti), prosleđuje virtuelni interfejs i pokreće sekvencu. Okruženje samo povezuje komponente. Sekvenca priprema jedan „paket" podataka kao što su dimenzije slike, pragove i samu sliku.

Drajver emulira dve memorije BRAM1 i BRAM2, učitava sliku u model memorije, upisuje ulazne parametre, daje impuls *start* i tokom rada DUT-a vraća podatke na **_indata* ili upisuje ono što DUT šalje na **_outdata*. Na kraju čeka da *ready* pređe u 1. Monitor posmatra interfejs gde na *start* signal beleži ulaze, a kada *ready* postane 1, uzima rezultat *objects_count_out* i šalje ga na poređenje u skorbord. Uz to prikuplja jednostavnu pokrivenost. Skorbord upoređuje dobijeni rezultat sa očekivanim i izveštava da li je test prošao ili nije. Interfejs spaja sve signale između DUT-a i testbenča i omogućava da drajver i monitor rade preko istog virtuelnog interfejsa.

Pored funkcionalne pokrivenosti, generiše se i HTML izveštaj iz simulatora, kako bi se videlo koliko su stanja i putanje FSM-a zaista obuhvaćeni testovima.

3.1 Projektovanje sekvenci, paketa i sekvencera

U UVM-u sekvence predstavljaju objekte u kojima se nalazi celokupna informacija o logičkim pobudama koje će biti prosleđene testiranom dizajnu (DUT-u). Sekvencer prosleđuje sekvence drajveru i kontroliše prosleđivanje jedne ili više sekvenci ka drajveru. Informacija koja se u jednom trenutku šalje naziva se paket.

U našem okruženju paket je klasa *control_path_txn* i predstavlja jednu potpunu obradu jedne slike. U tom objektu se nalaze sve informacije koje su drajveru i skorbordu potrebne da bi test imao smisla, dimenzije slike (*rows_in, cols_in*), pragovi boje (*lower_in, upper_in*), sama slika kao niz piksela (*image_data[],* 24-bitni RGB), kao i očekivani i dobijeni rezultat (*expected_objects i result_objects*). U kodu taj paket najčešće označavamo promenljivom *tr*, to predstavlja regulator ka jednoj instanci *control_path_txn*.

```
class control_path_txn extends uvm_sequence_item;

  // Ulazni signali
  rand bit [5:0] rows_in, cols_in;
  rand bit [23:0] lower_in, upper_in;
  rand bit start;

  // Ulazna slika (piksela)
  rand bit [23:0] image_data[];

  // Scoreboard poređenje
  rand bit [4:0] expected_objects;    // setuje se u sequence-u
       bit [4:0] result_objects;      // dolazi iz monitora (vif.objects_count_out)
```

Slika 2, Format paketa control_path_txn

Sekvenca *control_path_sequence* je jednostavni „single-shot“. Test je unapred kreiran i popunjava paket *tr* realnim vrednostima (dimenzije, pragovi, ulazna slika i očekivanje), a sama sekvenca, taj već pripremljen paket, samo preda drajveru preko standardnog UVM mehanizma (*start_item/finish_item*). Ovakav raspored pravi jasnu vezu gde jedan poziv sekvence odgovara jednoj slici i jednom rezultatu, što olakšava čitljivost logova i naknadnu analizu.

Sekvencer *control_path_sequencer* služi kao posrednik između sekvence i drajvera i parametrizovan je tipom *control_path_txn*. Pošto imamo samo jedan izvor pobude i jedan drajver, sekvencer nema posebnu ulogu osim da uredno isporuči paket drajveru čim ga sekvenca ponudi. Ovakva organizacija drži tok determinističnim, smanjuje šum u debagovanju i omogućava da se kasnije lako proširi, pokretanjem više „single-shot“ sekvenci sa različitim slikama.

3.2 Projektovanje driver-a

Uloga drajvera je da emulira signale koji će biti poslati uređaju. Direktno je povezan na interfejs od dizajna. Za razliku od sekvencera, koji se bavi slanjem podataka na visokom nivou, drajver treba da dobijene podatke na visokom nivou prosledi dizajnu koristeći postojeći interfejs. Drajver preuzima jedan pripremljen paket *control_path_txn* i emulira iz njega sve što stvarno okruženje oko DUT-a radi. Pre svega, kroz proceduru *preload_image()* učitava ulaznu sliku iz *tr.image_data* u *mem1*, dok *mem2* služi kao radni bafer za maske i privremene rezultate. Istovremeno drajver izvršava i proces *bram_model()* koji oponaša BRAM portove DUT-a. Kada DUT čita, drajver vraća podatke na **_indata*, a kada DUT piše, upisuje ih u odgovarajući niz *(mem1/mem2)*. Kada podaci postoje i interfejs je stabilan, program_*inputs()* zadaje dimenzije i pragove, generiše kratak impuls start i prepušta FSM-u da odradi svoj posao. Završetak jedne obrade detektuje se u *wait_done()*, gde drajver čeka *ready=1* uz određeni vremenski period proporcionalan broju piksela.

3.3 Projektovanje monitor-a

Monitor je pasivna komponenta i njegova uloga je da osluškuje i nadgleda signale. Sakupljene informacije o signalima prosleđuje ostalim komponentama. Dodatno, moguće je implementirati da se u njima vrši prikupljanje pokrivenosti i provera vrednosti. Monitor *control_path_monitor* posmatra upravljanje *start/ready* signala i BRAM aktivnosti. Na *start* uzima trenutne ulaze, dimenzije slike i pragove opsega boja, i čeka da DUT podigne *ready*.

Tada očitava *objects_count_out*, upisuje ga u polje *result_objects* istog paketa koji je prosleđen drajveru i šalje kompletan paket dalje ka skorbordu preko analitičkog porta (*ap.write*). Uz prenos rezultata, monitor prikuplja osnovnu funkcionalnu pokrivenost, da li je viđen konkretan par pragova sa očekivanim ishodom, kakva je bila raspodela čitanja/pisanja i opsega adresa na BRAM-ovima, kao i da li su se *start* i *ready* desili kada treba. Ovaj pristup daje jasnu sliku o tome šta su testovi zaista pokrili, bez uplitanja u sam DUT.

```
// 1) Funkcionalna pokrivenost: SAMO opseg za sliku 1 + rezultat
  covergroup cov_res;
    option.per_instance = 1;
    option.goal = 100;

    // uzorkuj samo ako je baš taj par pragova
    cp_lower : coverpoint s_lower iff (s_lower == 24'hB45A32) {
      bins img0 = {24'hB45A32};
    }
    cp_upper : coverpoint s_upper iff (s_upper == 24'hFF8246) {
      bins img0 = {24'hFF8246};
    }

    // očekuje se 1 objekat
    cp_objs : coverpoint s_objs iff (s_objs == 1) {
      bins one = {1};
    }

    // opcioni cross - potvrdi da je korišćen baš taj par + rezultat
    cx_thr_obj : cross cp_lower, cp_upper, cp_objs;
  endgroup

  // 2) BRAM aktivnosti
  covergroup cov_bram;
    option.per_instance = 1;

    cp_port : coverpoint s_port iff (s_port != 2'b00) {
      bins p1 = {2'b01}; // bram1
      bins p2 = {2'b10}; // bram2
    }

    cp_rw : coverpoint s_rw iff (s_port != 2'b00) {
      bins rd = {1'b0};
      bins wr = {1'b1};
    }

    cp_addr : coverpoint s_addr iff (s_port != 2'b00) {
      bins first = {12'd0};
      bins last  = {12'd3720};                  // 61*61 - 1
      bins low   = {[12'd1:12'd255]};
      bins mid   = {[12'd256:12'd2047]};
      bins high  = {[12'd2048:12'd3719]};
    }

    cx_bram : cross cp_port, cp_rw, cp_addr;
  endgroup

  // 3) Handshake start/ready
  covergroup cov_hs;
    option.per_instance = 1;
    cp_start : coverpoint s_start { bins pulse = {1}; }
    cp_ready : coverpoint s_ready { bins pulse = {1}; }
    cx_hs    : cross cp_start, cp_ready;
  endgroup
```

Slika 3, Funckionalna pokrivenost

3.4 Projektovanje scoreboard-a

Skorbord *control_path_scoreboard* je sudija koji na kraju obaveštava da li je test prošao ili nije. Svaku transakciju primljenu iz monitora poredi tako što uzima *tr.result_objects* iz DUT-a i upoređuje sa *tr.expected_objects* koje je postavio test. Ako se brojevi poklope, beleži prolaz, u suprotnom prijavljuje neusaglašenost sa jasnom porukom. Na kraju simulacije skorbord u *report_phase()* uzima sve rezultate i ispisuje koliko testova je validno od ukupnog broja. Proces verifikacije je završen i imamo informaciju o prolaznosti svakog testa.

3.5 Projektovanje agenta i konfiguracije

Agent umesto posebne UVM klase, realizovan je preko *sequencer, driver* i *monitor* koji su instancirani direktno u *control_path_env* i povezani TLM vezama. Funkcionalno to i dalje jeste jedan aktivni agent, samo bez dodatnog sloja apstrakcije. Prednost ovakvog načina projektovanja agenta je jednostavnost, jer imamo jedan izvor pobude, jedan drajver i jedan posmatrač, pa bi formalni agent bio čista forma bez praktične koristi. Ako se projekat proširi (više kanala, više DUT-ova, različiti režimi rada), lako je u sledećem koraku uvesti pravu agent klasu i prepakovati postojeće komponente bez promene ponašanja.

Konfiguraciju verifikacionog okruženja se svodi na *uvm_config_db*, umesto da se uvodi poseban *config* objekat. Test upisuje isti virtualni interfejs i u drajver i u monitor, i postavlja *expected_objects* koje monitor i skorbord koriste za poređenje. Tako se izbegava dupliranje stanja, jer sve ključne vrednosti dolaze iz jednog testa, komponente ostaju jednostavne i lako se ponovo koriste.

3.6 Projektovanje interfejsa

Virtuelni interfejs *control_path_if* predstavlja jedinstveno mesto na kome su objedinjeni svi pinovi dizajna i takt, kako bi UVM komponente mogle da komuniciraju sa DUT-om bez direktnog oslanjanja na HDL hijerarhiju. U njemu su grupisani upravljački signali za pokretanje i završetak obrade, parametri slike (dimenzije i pragovi boje), dva memorijska porta sa adresama i podacima za čitanje/pisanje, kao i izlazni brojač objekata. Interfejs se instancira u top testbenču zajedno sa DUT-om, zatim se njegov virtualni interfejs ubacuje u UVM konfiguracionu bazu i preuzima u drajveru i monitoru. Drajver, preko istog virtuelnog interfejsa, podesi ulazne parametre i tokom simulacije emulira BRAM portove, monitor samo pasivno osluškuje signale *start/ready*, prati čitanja/pisanja u memoriju i beleži izlazni rezultat.

3.7 Projektovanje test scenarija

U *build_phase* klase *control_path_test_base* najpre se kreira okruženje *control_path_env*, a zatim se iz *uvm_config_db* preuzima virtualni interfejs koji je ubacio top testbenč. Taj isti interfejs se prosleđuje i drajveru i monitoru kako bi obe komponente radile nad identičnim skupom DUT signala. Na ovom sloju fokus je napraviti okruženje i podeliti virtualni interfejs. Tek kada je okruženje kompletno i povezano, ima smisla definisati konkretan scenario.

Nakon toga se uvodi *control_path_test* kao konkretna instanca scenarija. U *build_phase* ovaj test u konfiguracionu bazu upisuje očekivanu vrednost broja objekata, kako bi je preuzeli monitor i skorbord. U *run_phase* test kreira sekvencu i jedan paket transakcije, popunjava paket realnim podacima (dimenzije slike, granice boje i ceo niz piksela učitan iz pomoćnog heder fajla), i startuje sekvencu. Tokom izvršavanja, test drži *run* fazu aktivnom da bi simulator imao dovoljno vremena da drajver postavi ulazne parametre i emulira BRAM, DUT obradi sliku i podigne *ready=1*, monitor preuzme rezultat i prosledi ga skorbordu, a skorbord ga uporedi sa očekivanom vrednošću. Na kraju, test završava *run* fazu i simulacija se uredno završava. Test ni u jednom trenutku direktno ne upravlja DUT-om, sav ulaz/izlaz ide kroz sekvencu, sekvencer, drajver i virtuelni interfejs, što scenario čini ponovljivim.

3.8 Projektovanje okruženja

Okruženje instancira i enkapsulira sve prethodno opisane komponente koje obavljaju operativni deo verifikacije, sekvencer, drajver, monitor i skorbord. U *build* fazi instancira četiri glavne komponente sekvencer, drajver, monitor i skorbord, a u *connect* fazi povezuje sekvencer i drajvera (drajver uzima po jedan paket kada je spreman) i *analysis* vezu preko koje monitor šalje gotove transakcije skorbordu na poređenje.

```
`ifndef CONTROL_PATH_ENV_SV
`define CONTROL_PATH_ENV_SV

`include "uvm_macros.svh"
import uvm_pkg::*;

`include "control_path_txn.sv"
`include "control_path_driver.sv"
`include "control_path_sequencer.sv"
`include "control_path_monitor.sv"
`include "control_path_scoreboard.sv"

class control_path_env extends uvm_env;
  `uvm_component_utils(control_path_env)

  control_path_sequencer seqr;
  control_path_driver    driv;
  control_path_monitor   mon;
  control_path_scoreboard sb;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);

    seqr = control_path_sequencer::type_id::create("seqr", this);
    driv = control_path_driver::type_id::create("driv", this);
    mon  = control_path_monitor::type_id::create("mon", this);
    sb   = control_path_scoreboard::type_id::create("sb", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);

    driv.seq_item_port.connect(seqr.seq_item_export); // sequencer → driver
    mon.ap.connect(sb.sb_port);                       // monitor → scoreboard
  endfunction

endclass

`endif
```

Slika 4, Environment modul

3.9 Projektovanje top modula

Testbenč *control_path_tb* predstavlja hardversko okruženje za simulaciju i prepušta kontrolu UVM-u. U njemu se generiše sistemski takt, zadaje inicijalni reset, instancira se DUT i interfejs i međusobno povezuju svi pinovi. Potom se virtuelni interfejs ubacuje u UVM konfiguracionu bazu, kako bi ga preuzele UVM komponente koje rade nad DUT-om i poziva

se *run_test*. Time top prepušta upravljanje UVM-u, drajver preko virtualnog interfejsa postavlja ulaze DUT-a i emulira BRAM čitanje/pisanje, monitor nadgleda i beleži rezultat, a skorbord upoređuje dobijeni izlaz sa očekivanim i prijavljuje da li je test prošao.

```
`timescale 1ns / 1ps

`include "uvm_macros.svh"
import uvm_pkg::*;

`include "control_path_if.sv"
`include "control_path_test.sv"

module control_path_tb;

  logic clk;
  control_path_if vif(clk);

  // Clock
  initial begin
    clk = 0;
    forever #10 clk = ~clk;
  end

  //reset
  initial begin
      vif.reset = 1;
      repeat (5) @(posedge clk);
      vif.reset = 0;
  end


  // UVM interface
  initial begin
    uvm_config_db#(virtual control_path_if)::set(null, "*", "vif", vif);
    run_test("control_path_test");
  end

  // DUT
  control_path dut (
    .clk                (vif.clk),
    .reset              (vif.reset),
    .start              (vif.start),
    .ready              (vif.ready),
    .rows_in            (vif.rows_in),
    .cols_in            (vif.cols_in),
    .lower_in           (vif.lower_in),
    .upper_in           (vif.upper_in),
    .bram1_en           (vif.bram1_en),
    .bram1_we           (vif.bram1_we),
    .bram1_addr         (vif.bram1_addr),
    .bram1_indata       (vif.bram1_indata),
    .bram1_outdata      (vif.bram1_outdata),
    .bram2_en           (vif.bram2_en),
    .bram2_we           (vif.bram2_we),
    .bram2_addr         (vif.bram2_addr),
    .bram2_indata       (vif.bram2_indata),
    .bram2_outdata      (vif.bram2_outdata),
    .objects_count_out(vif.objects_count_out)
  );

endmodule
```

Slika 5, Top modul

4. Prikupljanje pokrivenosti sistema

Pokrivenost je implementirana u *control_path_monitor* u 3 grupe, koje proveravaju pokrivenost rezultata obrade, pokrivenost aktivnosti nad BRAM portovima i pokrivenost

rukovanja signala *start/ready*. Monitor nije aktivna komponenta, on samo posmatra interfejs i u pravilnim trenucima uzorkuje signale i promenljive. Uzorkovanje je definisano kao:

1. Pokrivenost rezultata proverava da li je izvršen baš onaj scenario koji test očekuje. Prati se donja i gornja granica boje i broj prepoznatih objekata, i sve se uzorkuje isključivo za konkretan par pragova koji važi za sliku korišćenu u testu. Nakon pojedinačnih provera, proverava se i njihov *cross*, da li je upravo taj par pragova i upravo taj broj objekata viđen zajedno. Rezultat 100% znači da je ceo tok obrade slike zaista prošao od početka do kraja sa zadatim pragovima i da je DUT isporučio očekivan broj objekata.

Group - $unit_control_path_driver_sv_3208591166::control_path_monitor::cov_res

Group Info : cov_res

Score	Num Insts	Avg Insts Score	Weight	Goal	Merge Insts	Get Inst Crvg	Per Inst	Auto Bin Max	Print Missing	Comment
100	1	100	1	100	0	0	1	64	0	

Source File(s) : ../../../../../../verifikacija/control_path_monitor.sv

Group Instance[s] Info : 1 (Total)

Name	Score	Weight	Goal	At Least	Auto Bin Max	Print Missing	Comment
his .c ...	100	1	100	1	64	0	

Instance : his .cov_res

Instance Info : his .cov_res

Score	Weight	Goal	At Least	Auto Bin Max	Print Missing	Is Instantized	Comment
100	1	100	1	64	0	1	

Instance Variable[s] Summary : his .cov_res

Cover Points	Category	Expected	Uncovered	Covered	Avg Percent
3	Variables	3	0	3	100
1	Crosses	1	0	1	100

Instance Cover Point[s] Details : his .cov_res

Name	Expected	Uncovered	Covered	Percent	Goal	Weight	At Least	Auto Bin Max	Comment
cp_lower	1	0	1	100	100	1	1	64	
cp_upper	1	0	1	100	100	1	1	64	
cp_objs	1	0	1	100	100	1	1	64	

Instance Cross[es] Details : his .cov_res

Name	Expected	Uncovered	Covered	Percent	Goal	Weight	At Least	Comment
cx_thr_ ...	1	0	1	100	100	1	1	

Slika 6, cov_res pokrivenost

2. Pokrivenost aktivnosti nad memorijama ne meri sadržaj, već korišćenje memorijskih portova i zahvaćen opseg adresa. Beleži se koji je port bio aktivan (BRAM1 ili BRAM2), da li je operacija bila čitanje ili pisanje, i u kom se delu adresnog prostora pristup dogodio. Adresni prostor je podeljen na regione: prva lokacija, poslednja lokacija koja je stvarno korišćena u testu (3720 za sliku 61×61), i tri kontinualna podopsega između (*low, mid, high*). Na kraju se proverava i *cross* svih kombinacija *cp_port, cp_rw* i *cp_addr*. Rezultat 100% znači da je DUT, tokom jedne obrade i čitao i pisao oba porta i da je radio po celom adresnom opsegu od početka, kroz sredinu, do poslednjeg elementa. To potvrđuje da su sve faze toka (formiranje maske, dilatacija, erozija, kopiranja između bafera) zaista izvršene u punom opsegu, a ne samo na malom delu slike.

Group - $unit_control_path_driver_sv_3208591166::control_path_monitor::cov_bram

Group Info : cov_bram

Score	Num Insts	Avg Insts Score	Weight	Goal	Merge Insts	Get Inst Crvg	Per Inst	Auto Bin Max	Print Missing	Comment
100	1	100	1	100	0	0	1	64	0	

Source File(s) : ../../../../../../verifikacija/control_path_monitor.sv

Group Instance[s] Info : 1 (Total)

Name	Score	Weight	Goal	At Least	Auto Bin Max	Print Missing	Comment
his .c ...	100	1	100	1	64	0	

Instance : his .cov_bram

Instance Info : his .cov_bram

Score	Weight	Goal	At Least	Auto Bin Max	Print Missing	Is Instantized	Comment
100	1	100	1	64	0	1	

Instance Variable[s] Summary : his .cov_bram

Cover Points	Category	Expected	Uncovered	Covered	Avg Percent
3	Variables	9	0	9	100
1	Crosses	20	0	20	100

Instance Cover Point[s] Details : his .cov_bram

Name	Expected	Uncovered	Covered	Percent	Goal	Weight	At Least	Auto Bin Max	Comment
cp_port	2	0	2	100	100	1	1	64	
cp_rw	2	0	2	100	100	1	1	64	
cp_addr	5	0	5	100	100	1	1	64	

Instance Cross[es] Details : his .cov_bram

Name	Expected	Uncovered	Covered	Percent	Goal	Weight	At Least	Comment
cx_bram	20	0	20	100	100	1	1	

Slika 7, cov_bram pokrivenost

3. Pokrivenost rukovanja start/ready meri prisustvo impulsa *start* i događaja *ready*. Pojedinačni binovi za *start* i *ready* jesu pokriveni, jer drajver generiše impuls *start-a*, a DUT, nakon obrade, vraća *ready*=1. Međutim, *cross* ovih događaja u okviru *covergroup-a* podrazumeva istovremenu pojavu u istom ciklusu, što ne može da se desi čim je *start*=1, DUT napušta IDLE i spušta *ready*, a tek po završetku obrade *ready* ponovo postaje 1. Zbog toga je rezultat ove grupe 66,67%.

Group - $unit_control_path_driver_sv_3208591166::control_path_monitor::cov_hs

Group Info : cov_hs

Score	Num Insts	Avg Insts Score	Weight	Goal	Merge Insts	Get Inst Crvg	Per Inst	Auto Bin Max	Print Missing	Comment
66.6667	1	66.6667	1	100	0	0	1	64	0	

Source File(s) : ../../../../../../verifikacija/control_path_monitor.sv

Group Instance[s] Info : 1 (Total)

Name	Score	Weight	Goal	At Least	Auto Bin Max	Print Missing	Comment
his .c ...	66.6667	1	100	1	64	0	

Instance : his .cov_hs

Instance Info : his .cov_hs

Score	Weight	Goal	At Least	Auto Bin Max	Print Missing	Is Instantized	Comment
66.6667	1	100	1	64	0	1	

Instance Variable[s] Summary : his .cov_hs

Cover Points	Category	Expected	Uncovered	Covered	Avg Percent
2	Variables	2	0	2	100
1	Crosses	1	1	0	0

Instance Cover Point[s] Details : his .cov_hs

Name	Expected	Uncovered	Covered	Percent	Goal	Weight	At Least	Auto Bin Max	Comment
cp_start	1	0	1	100	100	1	1	64	
cp_ready	1	0	1	100	100	1	1	64	

Instance Cross[es] Details : his .cov_hs

Name	Expected	Uncovered	Covered	Percent	Goal	Weight	At Least	Comment
cx_hs	1	1	0	0	100	1	1	

Slika 8, cov_hs pokrivenost

5. Zaključak

U ovom projektu izvršena je funkcionalna verifikacija modula *control_path*, koji na osnovu zadatih pragova boje prepoznaje objekte na RGB slici i na izlazu ispisuje njihov ukupan broj.

Okruženje je zasnovano na UVM metodologiji sa jasno podeljenim ulogama gde virtuelni interfejs objedinjuje sve pinove DUT-a, preko njega drajver emulira BRAM portove i pokreće obradu, monitor posmatra dešavanja na interfejsu i beleži rezultat, a skorbord na kraju donosi *pass/fail* odluku. Pobuda je organizovana kao „single-shot“ sekvenca, jedan paket odgovara jednoj slici sa svojim dimenzijama, pragovima i očekivanim ishodom.

Test je koristio sliku 61×61 piksela, sa pragovima 0xB45A32–0xFF8246 i na izlazu je očekivao da se prepozna jedan objekat. Drajver je unapred učitao sliku u model memorije, programirao ulaze, generisao *start* i čekao *ready* uz zaštitni tajmaut, proporcionalan broju piksela. Monitor je na kraju proočitao *objects_count_out* i prosledio skorbordu, gde je on potvrdio podudaranje.

Model funkcionalne pokrivenosti pokriva rezultat obrade, aktivnosti nad memorijama i rukovanje *start/ready* signalima. *cov_res* je 100%, zato što je korišten baš zadati par pragova i dobijen je očekivan broj objekata. *cov_bram* je 100%, pošto je i čitanje i pisanje na oba porta kroz ceo relevantan adresni prostor, što potvrđuje da su sve faze toka stvarno prošle kroz celu sliku. *cov_hs* je 66,67% jer *cross* u covergroup-u traži da se *start* i *ready* signali dese u istom taktu, što protokol ovog DUT-a ne dopušta, to je očekivano i nije funkcionalni problem.

Funkcionalna verifikacija ovog sistema uspešno je sprovedena UVM metodologijom, a okruženje je moguće unaprediti gde bi kroz pokretanje regresionih testova proveravali više slika sa više različitih opsega boja.